Some thoughts on asynchronous API design in a post-async/await world (2016) (vorpus.org)

46 points by acjohnson55 on Oct 18, 2021 | hide | past | favorite | 18 comments

fouric on Oct 18, 2021 [–]

Can someone explain why the async keyword and the whole "red/blue functions" thing is necessary at all *in the source code*? Why can't the language implementation see that "await" is used in the body of a function, and automatically mark it as async?

jerf on Oct 18, 2021 | parent | next [–]

In general, it can.

In specific, backporting that onto languages that weren't designed to support that can be very difficult. There aren't very many languages using async/await that were designed that way from the beginning. I'm not sure I can name even one, actually. Many were over a decade old, and that is a lot of code set pretty deep into the concrete before the async/await machinery is installed.

You can look at Erlang, Haskell, and Go as things that just handles the matter internally, between the compiler and the runtime, and erases the color distinction. (Haskell adds back in plenty of color distinctions of its own, it is arguably the leader in "What if we had *dozens* of 'colors' of functions?" research, but "asynchronicity" is not one of them imposed by the runtime.)

I would certainly encourage any modern language designer starting from scratch to think long and hard before simply blindly porting async/await into their language. Look around at what else has been done first. I consider it an awfully klunky answer. I haven't looked into it personally but I hear Zig has an interesting answer.

pcwalton on Oct 19, 2021 | root | parent | next [–]

> In general, it can.

Yes, it's called threads.

> You can look at Erlang, Haskell, and Go as things that just handles the matter internally, between the compiler and the runtime, and erases the color distinction.

Erlang, Haskell, and Go use threads, just a particularly idiosyncratic M:N userspace implementation. There's no conceptual difference between what those three languages do and pthreads. Note that all three implementations are *stackful* (in Haskell's case, morally so rather than technically so).

> I would certainly encourage any modern language designer starting from scratch to think long and hard before simply blindly porting async/await into their language. Look around at what else has been done first. I consider it an awfully klunky answer.

Async/await allows you to have *stackless* coroutines, because of the explicit nature. This is a significant benefit, and dismissing it as "klunky" is a misunderstanding.

> I haven't looked into it personally but I hear Zig has an interesting answer.

Zig uses async/await with a global per-program switch that allows the whole program to be switched into blocking mode.

jerf on Oct 19, 2021 | root | parent | next [–]

"Async/await allows you to have stackless coroutines, because of the explicit nature. This is a significant benefit, and dismissing it as "klunky" is a misunderstanding."

No, I understand it perfectly. It is a disagreement about the relative cost/benefits of making the programmer do it vs. letting the runtime do it, even if it costs a bit.

I'd lay money 90%+ of the people doing async/await, either because they are stuck in a language in which it is the only choice (Javascript being the biggest entrant here since in its space there is no alternative), or because they thought they *needed* it, do not benefit from the additional 5-ish percent of performance gain, because programmers have been assuming they need the biggest, baddest toolset to write their code that handles a 50us request every few seconds since programming began. Instead they ought to be using a system that is far easier for the programmer and letting the runtime and compiler do the work.

It is good, even very good, that Rust provides the option to remove even those last few percent of overhead. It is not good that so many people who have problems that are orders of magnitude away from needing that level of performance choose something that requires more of the programmer instead of more of the compiler/runtime. That Rust community has decided this is almost the only way of doing concurrency (even if the Rust language itself technically is not requiring that) is one of the major things

keeping me away from it. I don't want to do that work unless I'm tight up against my CPU budget, and in a world of 64 core servers and when I'm not doing any hard realtime networking like routing, none of my tasks are even close to requiring that. (Not that I *use* those 64 core servers, I'm just saying they're there before I need to go crazy to get the last few percent. I'm usually more in the position of explaining to my coworkers that the system they seem to think needs a 16core machine with oodles of RAM runs rather comfortably on the second smallest instance there is and is still mostly twiddling its thumbs.)

I don't *care* about that last little bit of performance in my personal need space, and even the *vast majority* of Rust users shouldn't either. Again, that it's *available* for the ones who legitimately do is very good. But it's a klunky, ugly default to go slightly faster, or to enable the average programmer to fit in to 1% of the available RAM for their task instead of 1.4%, to have to add a color to all your functions and tell the compiler over and over again "Here's where you split my function to do some IO here... and I do some here... and I do some here... and I do some here... and I do some here..." unto literally thousands of times for even medium sized programs.

It's good that it exists. It's a *bad default*.

toast0 on Oct 19, 2021 | root | parent | prev | next [–]

Erlang doesn't have function colors because every function is async. Which is one way to do it. You can build synchronous feeling functions by waiting for a response. Execution can be suspended at any function call, which includes at least some operators (! (send) and -- (list subtract sort of) as two operators I'm familiar that are likely to suspend), but no loops means you need to use recursion to loop which means you need to call functions which means there will be a bounded time until you hit a suspend point.

But it's pretty hard to add those constraints to an exisiting language if it's not already pretty close.

dragonwriter on Oct 19, 2021 | root | parent | next [–]

> Erlang doesn't have function colors because every function is async.

All Erlang functions are synchronous, not async. (spawn, and variants thereof, run *another* otherwise synchronous function in a new logically parallel context to and from which asynchronous signals can be sent and received, which is kind of like async with superpowers, but in a way which does not create colored function issues.)

jerf on Oct 19, 2021 | root | parent | next [–]

In the somewhat broken sense in which "async" and "sync" are now used courtesy of Node's propagandization of the terms, all Erlang functions are async. Any of them can do some IO, and the runtime will switch away from the Erlang process to do the IO and run other Erlang processes, and then once the IO is done it automatically comes back to finish the process' task.

I call it somewhat broken because this sense conflates the question of how the code is written with how the code is executed, which creates a semantic blind spot in which a programmer that has only used this paradigm becomes incapable of conceptualizing a runtime system in which the code is written "synchronously" but actually run "asynchronously". But it's a popular definition and you need to understand that people use the term to mean both the fact that a function is written with explicit "async/await" annotations and that it is run that way, and that you need to very carefully explain that these two things are not actually as tied together as they think because it is a foreign thought if that is all they know, which is large and probably growing number of programmers right now.

I don't mean anything *mean* by that statement, I'm just being descriptive. I remember my days of thinking line numbers in programs were fundamental and it would be impossible to conceive of a programming language lacking them.

dragonwriter on Oct 19, 2021 | root | parent | next [–]

> In the somewhat broken sense in which "async" and "sync" are now used courtesy of Node's propagandization of the terms, all Erlang functions are async.

> Any of them can do some IO, and the runtime will switch away from the Erlang process to do the IO and run other Erlang processes, and then once the IO is done it automatically comes back to finish the process' task.

I don't think that's the common use, nor is it particularly *true* even in that use, since Erlang’s main runtime is M:N threaded so that up to the limits of parallelism on the host machine (or the parallelism limit set when starting the runtime), Erlang processes are parallel to start with, though it is true that the runtime will *also* use IO and certain other opportunities to swap the running Erlang process in a given native thread when at the parallelism limit.)

It's worth noting that in that (badly broken) definition, all *Ruby* and *Python* functions are async (because the runtime, even with the GIL/GVL, swaps running threads on IO and certain other opportunities), which is particularly confusing in Python's case, because Python also has colored functions with async/await (that is, Python has the kind of “async” that has

generally defined the term, especially in the context of colored functions, since C# popularized the term.)

It is true there is another *older* use of the term, which remains popular in the absence of the "colored function" context which also applies to Node, but that definition doesn't refer to when the runtime takes the opportunity to switch tasks but instead refers to having nonblocking operations (mainly IO) with callback/promise (or, later, async/await) semantics allowing concurrency within a single logical thread/process of execution.

Ruby, in that sense has async libraries despite having not having sync/await syntax. Python had async libraries before async/await, Node is all async even before async/await, and Erlang is still entirely synchronous, relying on explicit separate logical processes for both concurrency and parallelism.

heavyset_go on Oct 18, 2021 | root | parent | prev | next [–]

> *I haven't looked into it personally but I hear Zig has an interesting answer.*

Can you expound on this or share any links about it?

lioeters on Oct 19, 2021 | root | parent | next [–]

I'm curious about this also. From what I could find about how async/await works in Zig..

> The style of Zig's async may be described as suspendible stackless coroutines. Zig's async is very different to something like an OS thread which has a stack, and can only be suspended by the kernel.

> Furthermore, Zig's async is there to provide you with control flow structures and code generation; async does not imply parallelism or the usage of threads.

https://ziglearn.org/chapter-5/

rdw on Oct 19, 2021 | parent | prev | next [–]

People care about performance a lot, especially when writing evented I/O. The explicitness is pretty useful at preventing unexpected performance cliffs.

One of the main pieces of feedback I heard about green threads is that they are "just as confusing" as threads. Developers tend to view preemptive multitasking as a difficult-to-manage hazard. Non-explicit async is not technically preemptive, but, most developers won't know which functions are going to cause a context switch, and it can change from one version to the next. So they kinda end up assuming they're all gonna context switch, so they put mutexes on things and basically do all the tedious and annoying work that threading entails. It's simultaneously "too much magic" and "not enough magic".

That said, Ruby's new green threading stuff [1] is very exciting, has the potential to revitalize the concept, and we'll just have to see how things turn out in the next five years.

[1] http://www.wjwh.eu/posts/2020-12-28-ruby-fiber-scheduler-c-e...

pansa2 on Oct 19, 2021 | parent | prev | next [–]

Eric Lippert explains the reasons why `async` was chosen for C# here: https://docs.microsoft.com/en-au/archive/blogs/ericlippert/a... . Other languages have used the same keyword either for their own good reasons or just because they wanted to copy what was being used elsewhere.

Note that even without `async`, you'd still have red and blue functions (those which contain `await` and those which don't), it would just be harder to tell which was which.

atarian on Oct 18, 2021 | parent | prev | next [–]

I imagine it's not as easy as it looks. Not only do you have to keep track of all the async functions, you need to keep track of the non-async functions that call those async functions, which would likely be a runtime check.

tshaddox on Oct 19, 2021 | parent | prev | next [–]

I assume that's possible in many programming languages, although may be a pain for the parser. Doesn't Python already totally change how it parses method definitions if there's a single `yield` keyword in the definition? That seems pretty similar to me.

zffr on Oct 19, 2021 | root | parent | next [–]

It's also possible to keep the parser simple, and look for "await" function calls during static analysis of the AST.

magicalhippo on Oct 18, 2021 | parent | prev | next [–]

I think this is a bit like the dynamic vs static typing discussion that's currently raging in another topic[1].

Being explicitly marked makes it self-documenting. It allows someone to see just the definition to know that it is async.

This is especially useful in conjunction with interfaces (or pure virtual base classes).

In addition there's function pointers to consider.

[1]: https://news.ycombinator.com/item?id=28907154

XorNot on Oct 19, 2021 | root | parent | next [–]

Isn't this what a type system will tell me though? If I call a function and get back a promise of a value, then I know I need to do more work to get the value.

8note on Oct 19, 2021 | parent | prev [–]

That covers one direction of errors, putting an await on a sync function, but doesn't ensure that you put an await on async functions when needed.

Async specifies that a function has two different behaviours: starting and finishing. You could assume everything is async, in which case the problem goes away, but that also moves the problem from code and into your head to remember whether something can be just started without completing

Guidelines | FAQ | Lists | API | Security | Legal | Apply to YC | Contact

Search: